

Microarchitecture level

Microarchitecture level

The main job of the microarchitecture level is to create and implement the Instruction Set Architecture (ISA) using digital logic components.

Where its design is dependent on the ISA and the cost and performance goals of the computer. Many modern ISAs, like RISC have simple instructions that can usually be executed in a single clock cycle.

While more complex ISAs, like the one in the Core i7, have instructions that can take multiple clock cycles to execute.

Microarchitecture level

The same ISA can be implemented by many different microarchitectures, each making different cost/performance tradeoffs.

- Intel and AMD both build CPUs that implement the **x86-64** ISA, so software compiled for one runs on the other.

But **internally**, an Intel Core and an AMD Zen chip have

- completely different *microarchitectures*
- different pipeline depths,
- different cache hierarchies,
- different execution unit counts

while presenting the *exact same* instruction set to the programmer.

the ISA is a **contract** with software, while the microarchitecture is free to change under the hood as long as it honors that contract.

Example

Because there are no general principles of microarchitecture design, as each ISA has its own unique features and requirements, we will instead look at an example.

Our main example for this section of the course will be a subset of the Java Virtual Machine which only contains integer instructions

We'll name it **IJVM** (Integer Java Virtual Machine).

Example

The real JVM is a **stack machine**

Instead of operating on named registers like `R1` or `R2`, most instructions push and pop values from an operand stack.

- `IJVM` keeps this stack-machine style, but strips out everything related to objects, floating point, exceptions, and threads, leaving just enough (integer arithmetic, branches, simple method calls) to demonstrate how a microarchitecture executes an ISA.
- It's small enough to fully discuss in a course, but realistic enough that the same ideas apply directly to real CPUs like x86 or ARM.

Example

Our microarchitecture will contain a **microprogram** whose job is to fetch, decode, and execute IJVM instructions.

For example, if in the instruction set level, the line

```
1  ADD R1, R2, R3
```

Is written, the microprogram will **execute** a sequence of instructions using digital logic components to perform the operation.

Theoretically speaking, after understanding microarchitecture, you should be able to buy a bag of transistors and a breadboard and build the IJVM physically.

Microprogram

We can think of the **microprogram** as a *programming problem*, where each instructions at the ISA level is a *function* to be called by a master program.

Our microprogram would have a set of variables called **state**. Where each function of our microprogram would change at least some part of the state.

For example, a **Program Counter** is part of the state which indicates the location of the next instruction/function to be executed. And during each instruction, the Program Counter is updated to point to the next instruction.

Microprogram

Besides the Program Counter (PC), a typical microarchitecture's state includes several other named registers, each with a specific job:

Register	Full name	Purpose
PC	Program Counter	Address of the next instruction to fetch
MAR	Memory Address Register	Holds the address about to be sent to memory
MDR	Memory Data Register	Holds the data being read from or written to memory
SP	Stack Pointer	Points to the top of the operand stack
MBR	Memory Buffer Register	Holds the most recently fetched instruction byte

Every one of these is just a plain register in the data path, updated according to the rules the microprogram defines for each instruction

Microprogram

The `IVJM` instructions we'll be dealing with are fairly short, usually containing one or two fields.

The first field is the **opcode** which indicates the operation to be performed, like `ADD` or `BRANCH`. And the following fields are usually called **operands**.

Operands identifies which variable, which register, or which memory location the operation is to be performed on.

This model of execution is often called the

```
1  fetch-decode-execute cycle
```

And is a useful abstraction to the implementation of a microprogram.

Microprogram

Suppose the opcode byte `0x60` means `ADD` (pop two values off the stack, push their sum):

1. **Fetch** - the byte at address `PC` is read into `MBR`, and `PC` is incremented
2. **Decode** - the microprogram recognizes `0x60` and dispatches to the routine that implements `ADD`
3. **Execute** - the top two values are popped from the stack (via `SP`), added together in the ALU, and the result is pushed back onto the stack

This exact cycle `fetch, decode, execute` repeats for every instruction

Data Path

Data Path

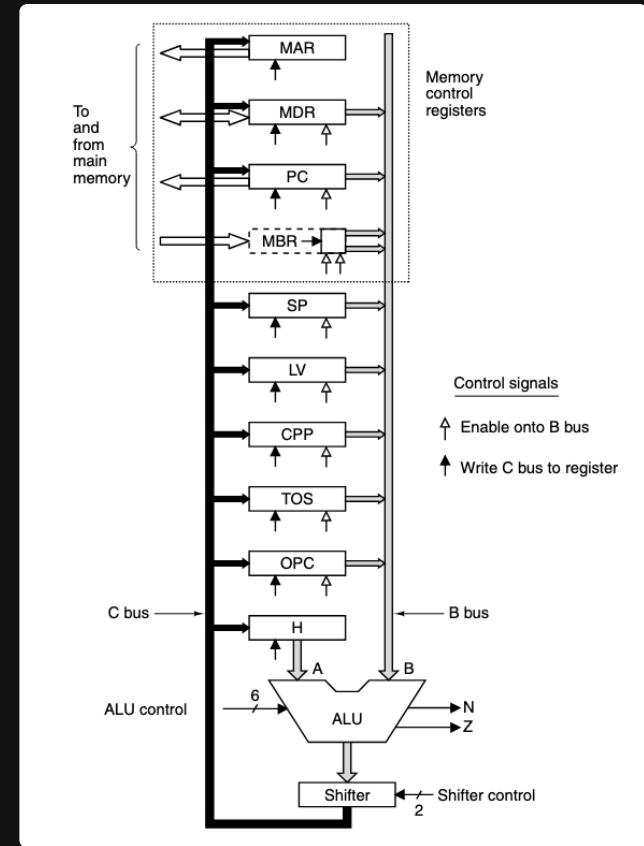
The data path is the part of the CPU which has the *ALU, inputs, and outputs*.

On the right is the data path for our example IJVM microarchitecture, which is fairly similar to the data paths of most machines.

It contains a few 32 bit registers with *symbolic names* (MAR, MDR, PC, etc), and most of these can *drive* their contents onto the B bus

The output of the ALU drives the shifter, and the C bus, who's value can be written into one or more registers

Note that there is no A bus



Data Path

In many real data paths, both ALU inputs come from a bus (an "A bus" and a "B bus"), so any two registers can be combined freely.

Our simplified IJVM data path fixes the ALU's left input to always come from a special latch (`H`), rather than a full bus. This means:

- Any register that needs to be the *left* operand must first be copied into `H`
- Only the *right* operand can be driven directly from a register through the `B` bus

This is a deliberate simplification that keeps the example easier to follow, at the cost of sometimes needing an extra cycle to shuffle a value into `H` before it can be used.

Real CPUs often do have full dual-bus (or wider) input networks precisely to avoid this extra step.

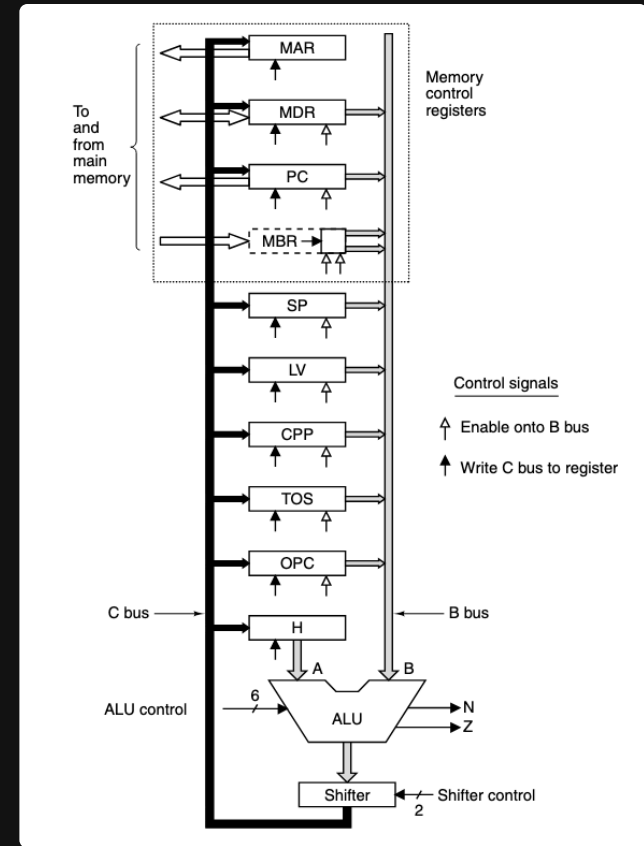
Data Path

The ALU itself is identical to the one we saw before. But the functions are determined by six control lines

2 data lines (F_0 , F_1) and 4 condition lines ENA (Enable A) , ENB , $INVA$ (Invert A) , $INVB$,

Note that addition in this case is arithmetic addition, not boolean addition.

F_0	F_1	ENA	ENB	INVA	INC	Function
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
1	0	1	1	0	0	$\bar{B}$
1	1	1	1	0	0	$A + B$
1	1	1	1	0	1	$A + B + 1$
1	1	1	0	0	1	$A + 1$
1	1	0	1	0	1	$B + 1$
1	1	1	1	1	1	$B - A$
1	1	0	1	1	0	$B - 1$
1	1	1	0	1	1	$-A$
0	0	1	1	0	0	$A \text{ AND } B$
0	1	1	1	0	0	$A \text{ OR } B$
0	1	0	0	0	0	0
1	1	0	0	0	1	1
1	1	0	0	1	0	-1



Data Path

ENA	ENB	INVA	INVB	F1 F0	Result
1	1	0	0	01 (ADD)	$A + B$
1	0	0	0	01 (ADD)	A (pass-through)
1	1	0	1	01 (ADD)	$A - B$
0	1	0	0	11 (AND handled elsewhere)	isolates B

Data Path Timing

Data Path Timing

It's possible to read and write the registers in *one cycle*,

For example, you can

- put the contents of the SP register on the B bus
- disable the ALU's left input
- enable the INC signal
- and store the result in SP, incrementing SP by 1

All in a *single cycle*

To do this without losing data and producing garbage. We perform the operations at different times within the cycle.

This is called *Data Path Timing*

Data Path Timing

It would certainly be *simpler* to split "read SP" and "write SP" into two separate clock cycles

Many simple microcontrollers do exactly that.

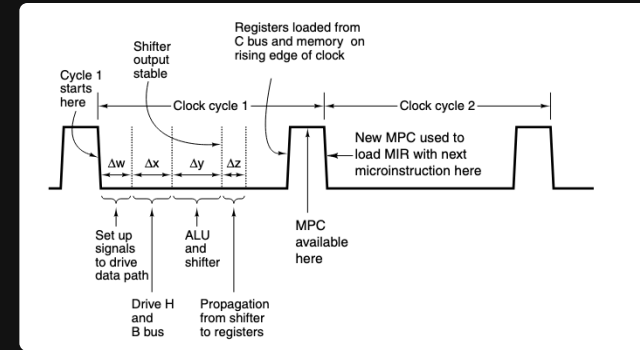
But doing both in one cycle roughly **halves** the number of cycles needed for register-update instructions, which matters a lot for something as frequently executed as incrementing the stack pointer.

The tradeoff is added design complexity: signals must be carefully staged within the cycle so that the *read* of the old value always happens before the *write* of the new value reaches the same register

Data Path Timing

On the right is a timing diagram for the data path of the previous example

1. A short pulse is produced at the start of each clock cycle
2. On the falling edge, the bits that will drive the gates are set up. This takes some time called Δw
3. The **H** register drives the data into the **B** bus taking Δx time
4. The ALU disables the left input, enables the incrementer, and produces the result on the **C** bus taking Δy time



5. And an additional Δz time is taken to propagate along the C bus and to the registers so that they can be loaded on the the rising edge of the next pulse if needed

Data Path Timing

The whole point of tracking Δw , Δx , Δy , and Δz separately is to guarantee the **total delay fits inside one clock cycle**:

$$T_{cycle} \geq \Delta w + \Delta x + \Delta y + \Delta z$$

Suppose

- $\Delta w = 1$ ns,
- $\Delta x = 2$ ns,
- $\Delta y = 3$ ns, and
- $\Delta z = 1$ ns.

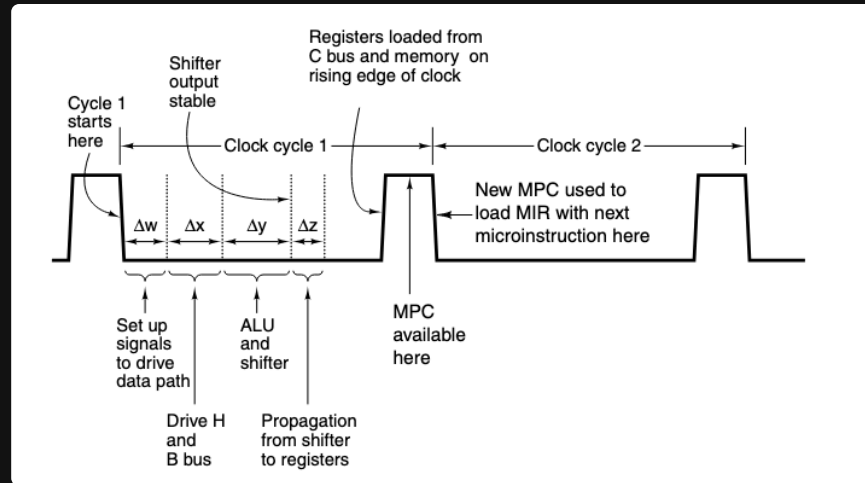
The minimum clock period is $1 + 2 + 3 + 1 = 7$ ns, giving a maximum clock frequency of roughly $1/7\text{ns} \approx 143$ MHz. If any single component gets slower (e.g. a longer C bus increases Δz), the *entire* CPU's maximum clock speed drops, since every cycle must accommodate the slowest path.

This is exactly the same "critical path" idea used to determine the maximum clock speed of real CPUs today.

Data Path Timing

This type of process requires *rigid timing*, *long clock cycles*, *a known minimum propagation time*, and *fast loading of the registers*.

But with good engineering, the data path can be made to work extremely reliably. And actual machines work the exact same way, with more complex data paths and timing diagrams.



Note that this works through *implicit* subcycles, meaning no additional clocks and sub clocks are needed. It works similarly to asynchronous circuits, but within a single cycle of a synchronous clock.

Data Path Timing

It's worth being precise about the comparison drawn above:

- A **truly asynchronous** bus (as seen in the Buses lecture) has no clock at all; each transfer is driven entirely by handshake signals like $\overline{MSYN}$, $\overline{SSYN}$.
- The data path's **implicit subcycles** still happen inside a single synchronous clock edge-to-edge period. There's no separate clock for Δw , Δx , Δy , Δz , they're just the natural propagation delays of signals moving through wires and gates, which the designer accounts for when picking the clock period.

The *clock* is synchronous, but the *internal ordering* of events within one clock cycle relies on physical propagation delay, the same underlying phenomenon that makes fully asynchronous buses work just constrained to fit inside a single tick instead of being open-ended.